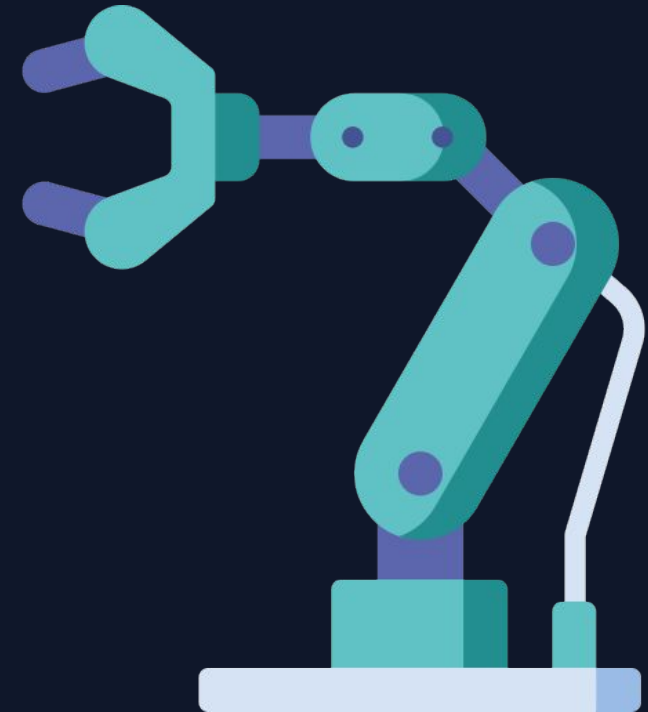
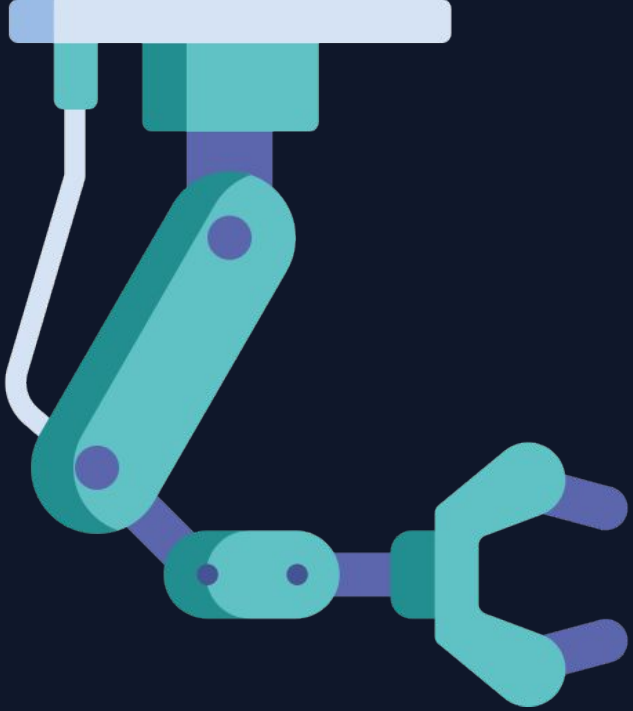
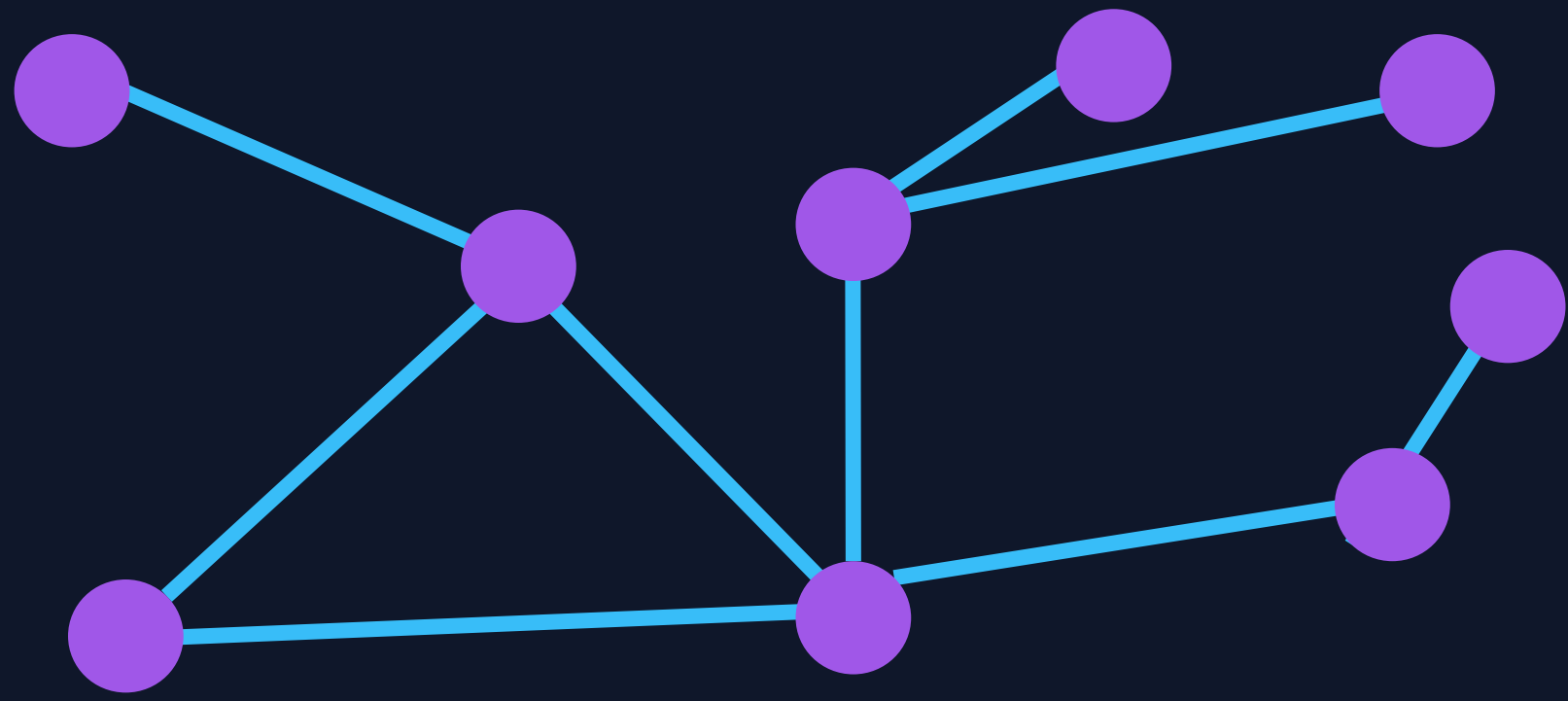


Robotic Operative System ROS 2

Introducción y
Taller Práctico de Robótica



QUE ES ROS?



QUE ES ROS?

HARDWARE DEL ROBOT



ROS 2 (Robot Operating System)

ROS
(Robot Operating System)

SOFTWARE Y ALGORITMOS (NODOS)



El 'sistema nervioso' que conecta y gestiona la comunicación entre todos los componentes del robot

CONCEPTOS FUNDAMENTALES



NODOS

Procesos individuales que realizan tareas específicas (ej. leer cámara).



TÓPICOS

Bus de datos streaming (Publicador / Suscriptor).



SERVICIOS

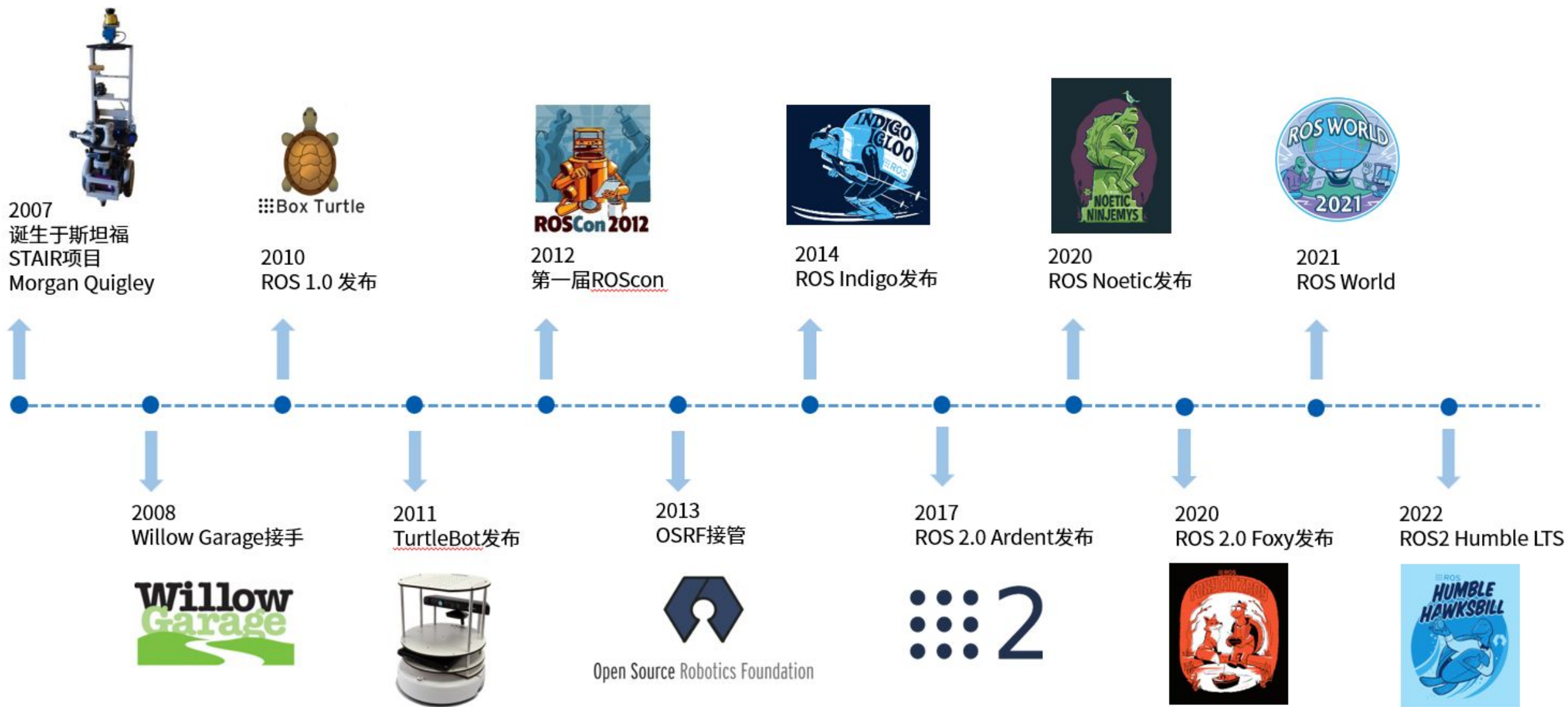
Comunicación síncrona "Cliente/Servidor".



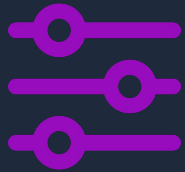
ACCIONES

Metas largas con feedback y cancelación.

QUÉ ES ROS?



CONCEPTOS FUNDAMENTALES



PARAMETROS

Valores específicos para asignar a variables internas a la hora del inicio del programa. (ej `config.yaml`)



ARCHIVOS LAUNCH

Archivos en formato `.xml` o `.py` que gestionan la inicialización secuencial de todos los programas con los parametros especificos

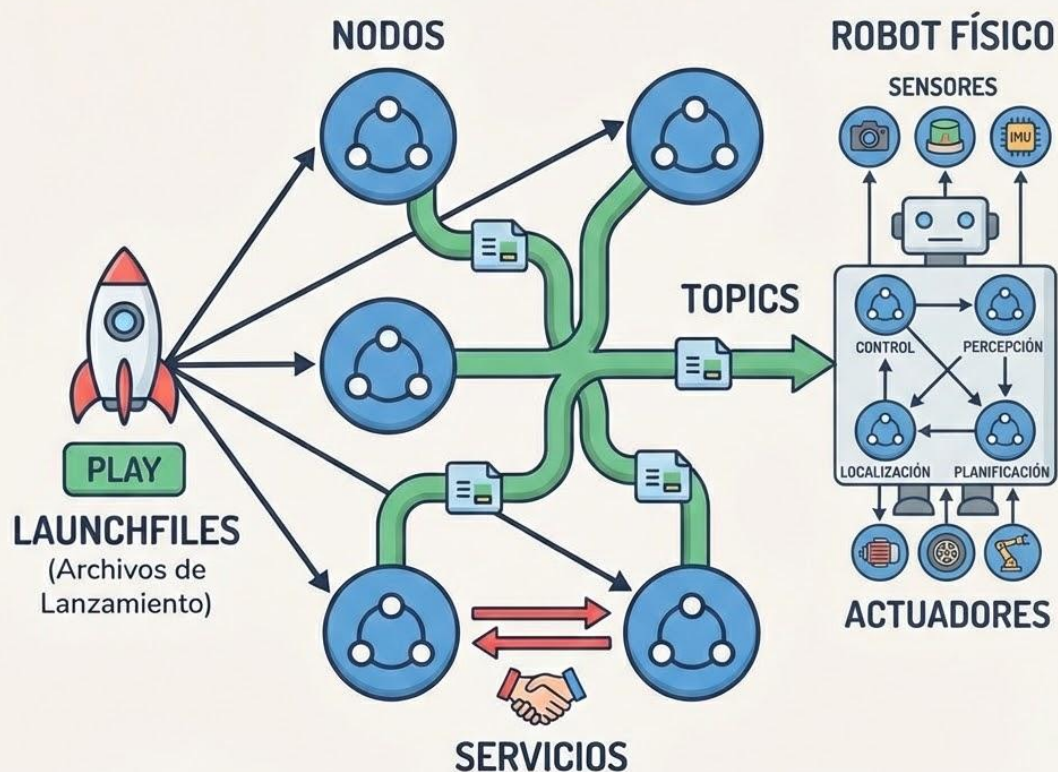


ROSBAG

Proceso externo que permite registrar todos los datos, mensajes y situaciones que se han dado durante la grabación. Ideal para debugear y recrear situaciones reales

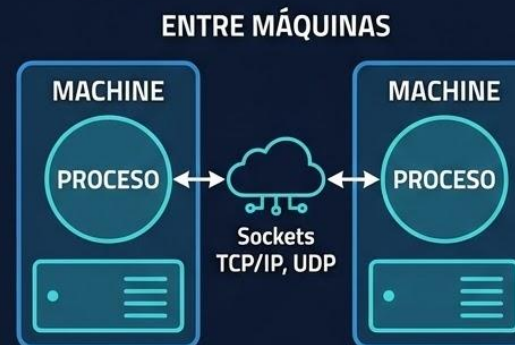
CONCEPTOS FUNDAMENTALES

ECOSISTEMA ROS



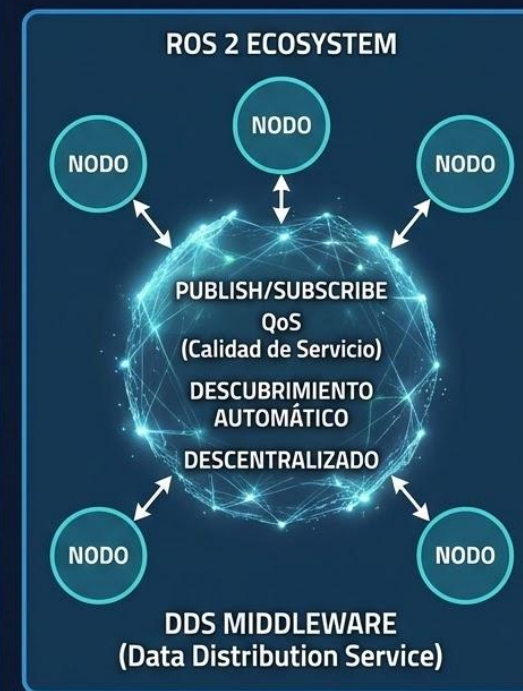
COMPARACIÓN: ROS 2/DDS vs. CONVENCIONAL

COMUNICACIÓN CONVENCIONAL (IPC/RED)



Acoplamiento directo, punto a punto, dependiente del OS.

ROS 2 & DDS (Data Distribution Service)



Desacoplamiento, comunicación basada en datos, interoperable.

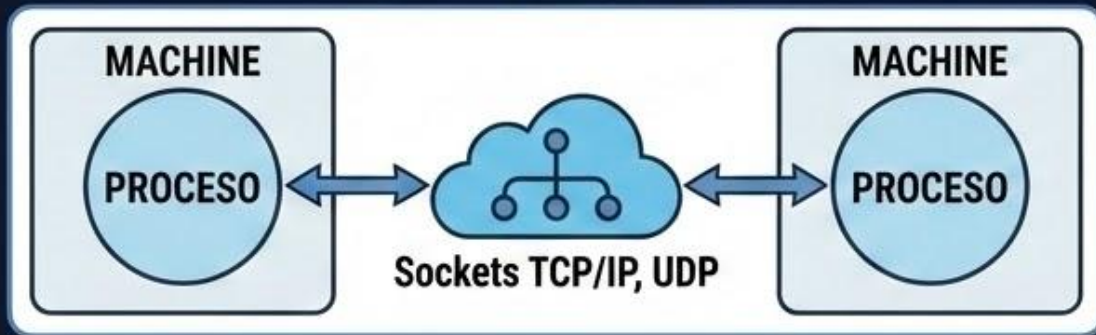
COMPARACIÓN: ROS 2/DDS vs. CONVENCIONAL

COMUNICACIÓN CONVENCIONAL (IPC/RED)

MISMA MÁQUINA (IPC)



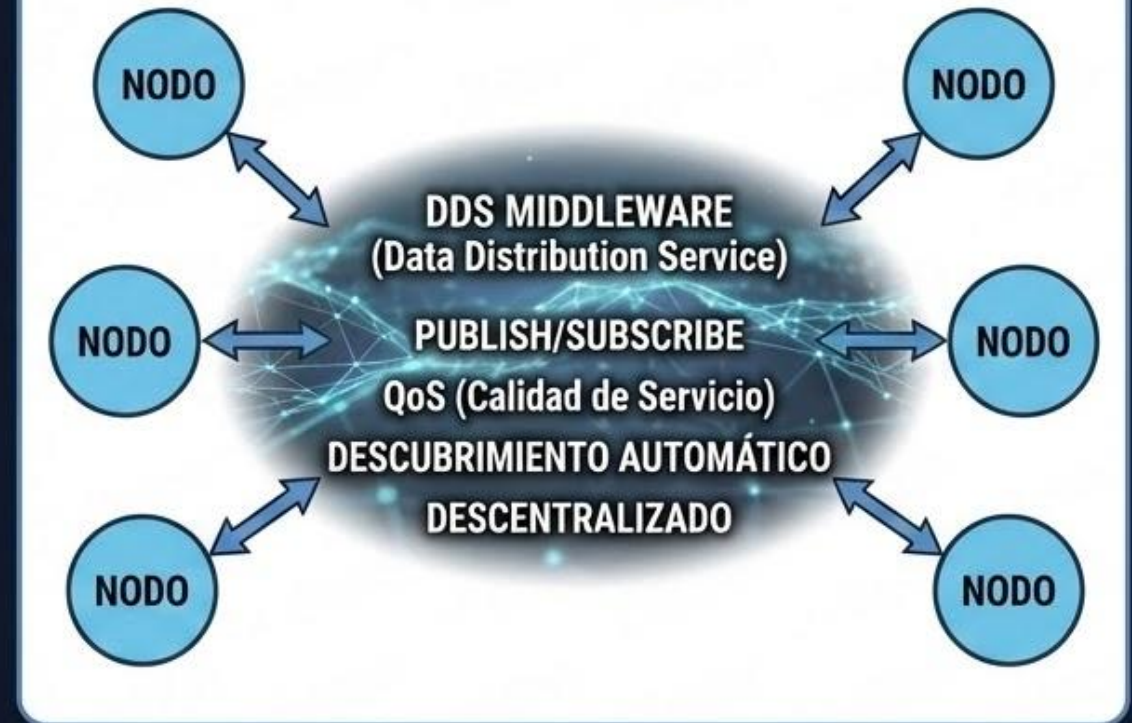
ENTRE MÁQUINAS



Acoplamiento directo, punto a punto,
dependiente del OS.

ROS 2 & DDS (Data Distribution Service)

ROS 2 ECOSYSTEM



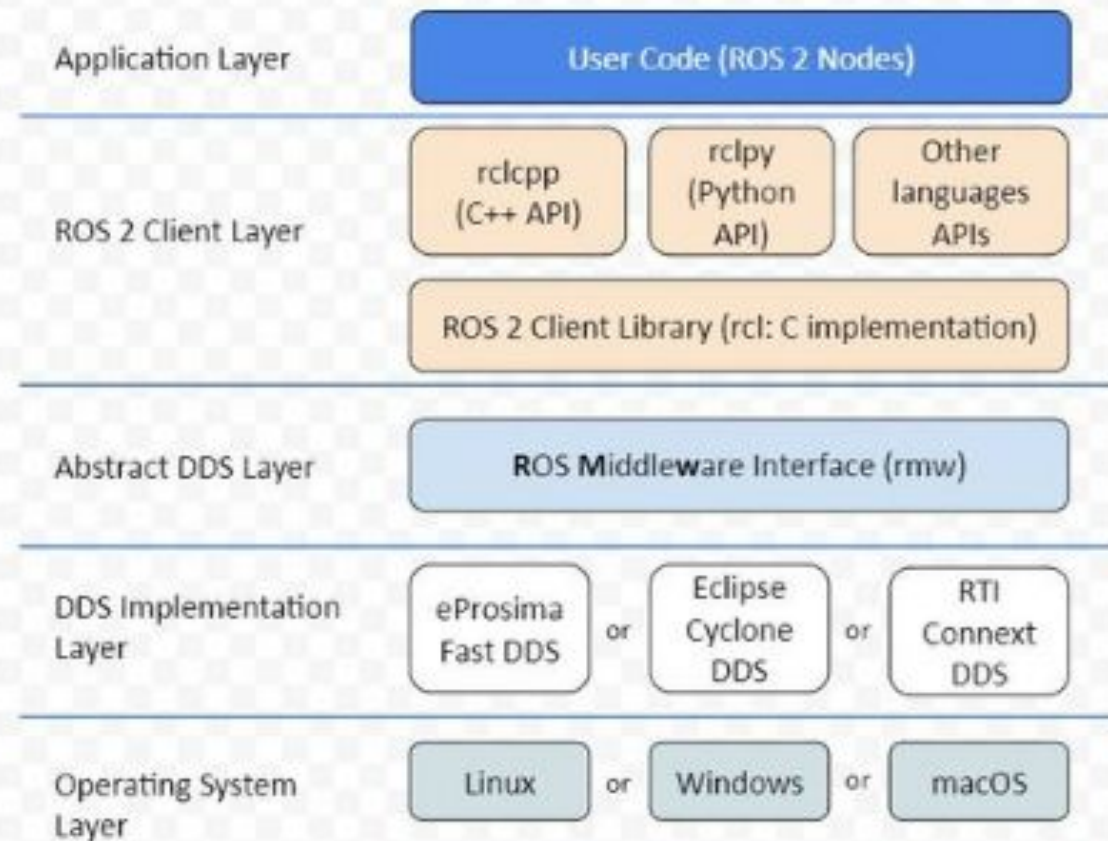
Desacoplamiento, comunicación basada en
datos, interoperable.

ARQUITECTURA ROS 2

- > **Aplicación:** Código C++ / Python.
- > **rclcpp/rclpy:** Librerías cliente estándar.
- > **RMW:** Capa de abstracción intermedia.
- > **DDS:** Estándar de comunicación industrial.

Diseñado para tiempo ejecución en tiempo real

ROS 2 Architecture Overview

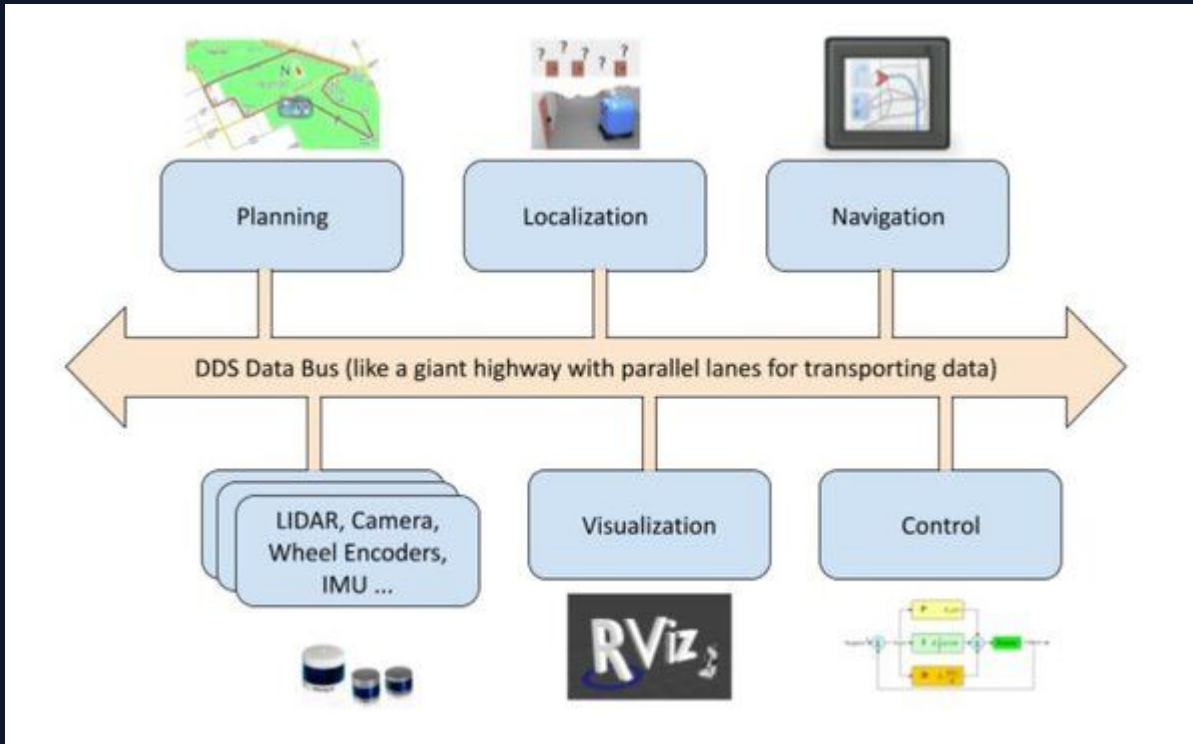


DDS = Data Distribution Service is a decentralized, publish-subscribe communication protocol.

rmw = ROS MiddlewAre Interface hides the details of the DDS implementations.

Use rclcpp for efficiency and fast response times, use rclpy for prototyping and shorter development time.

¿QUÉ ES EL DDS?



DATA DISTRIBUTION SERVICE

Es el "cartero" inteligente de ROS 2. A diferencia de ROS 1, no necesita un servidor central (Master).

Descubrimiento Automático: Los nodos se encuentran solos en la red.

QoS (Quality of Service):

Configurable.

- > Reliable: Garantiza entrega (TCP-like).
- > Best Effort: Prioriza velocidad (UDP-like).

Distribuido: Si un nodo cae, el resto sigue funcionando.

FORMATO DE MENSAJES

TIPOS DE INTERFACES (ARCHIVOS .MSG, .SRV, .ACTION)

.msg (Tópicos)

```
geometry_msgs/Pose.msg:  
Point position, Quaternion orientation
```

.srv (Servicios - Request/Response)

```
example_interfaces/AddTwoInts.srv:  
int64 a, int64 b  
--- int64 sum
```

.action (Acciones - Goal/Result/Feedback)

```
nav2_msgs/NavigateToPose.action:  
  PoseStamped goal  
--- std_msgs/Empty result  
--- float32 distance_remaining
```

ESTÁNDARES Y TIPOS DE DATOS

BASE: IDL (Interface Definition Language)

Estándar de definición de interfaces de la OMG, base para la compatibilidad DDS. Define la estructura de los datos de forma independiente del lenguaje.

TIPOS PRIMITIVOS COMPATIBLES

bool	uint16
byte	int32
char	uint32
float32	int64
float64	uint64
int8	string
uint8	arrays[]
int16	

EJEMPLO

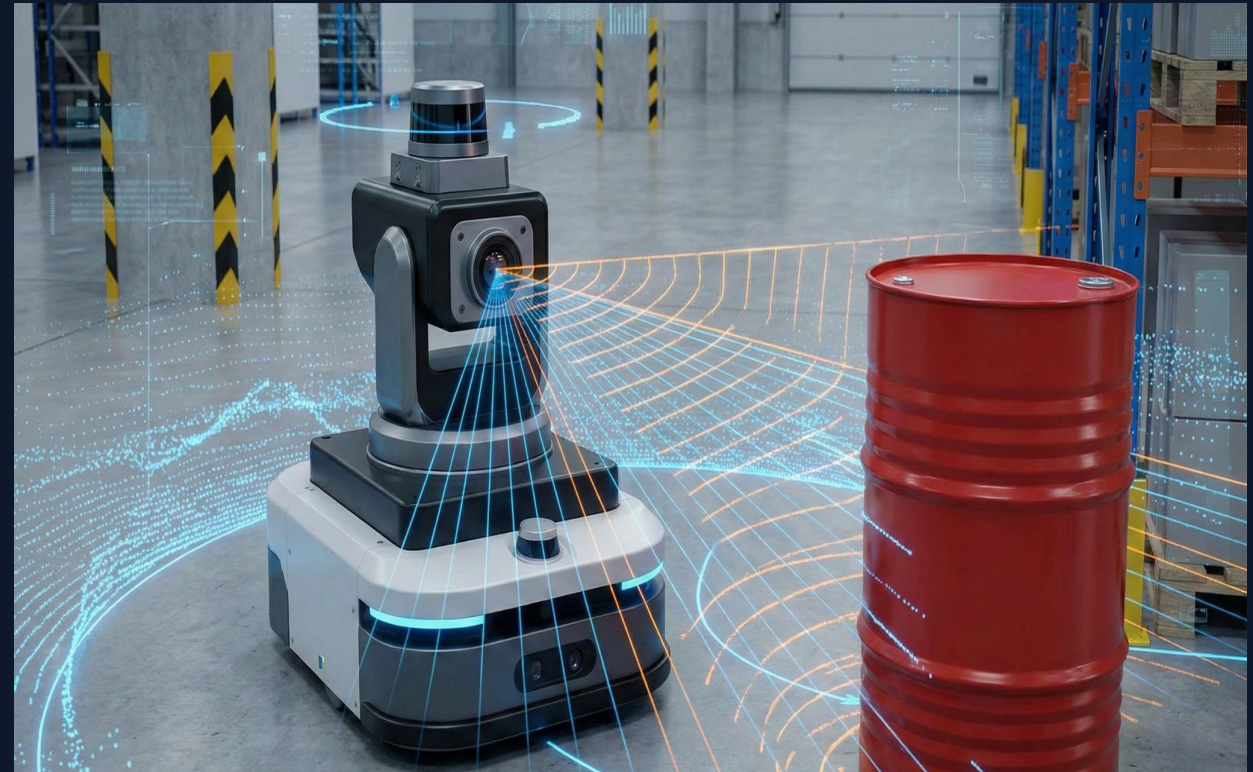
FLUJO DE DATOS: EVASIÓN DE OBSTÁCULOS

Imaginemos cómo se conectan los nodos para que un robot no choque.

```
[Lidar] --> /scan --> [Seguridad]
[Cámara] --> /image --> [Visión IA]

[Seguridad] + [Visión IA] --> /cmd_vel --> [Motores]
```

Cada caja es un nodo independiente. Si el nodo de visión falla, el nodo de seguridad puede seguir deteniendo el robot.



ECOSISTEMAS DE SIMULACIÓN



GAZEBO (CLASSIC / IGNITION)

- Estándar de la industria
- Física robusta
- Integración nativa perfecta



NVIDIA ISAAC SIM

- Fotorrealismo (RTX)
- Entrenamiento de IA (Isaac Gym)
- Omniverse



WEBOTS

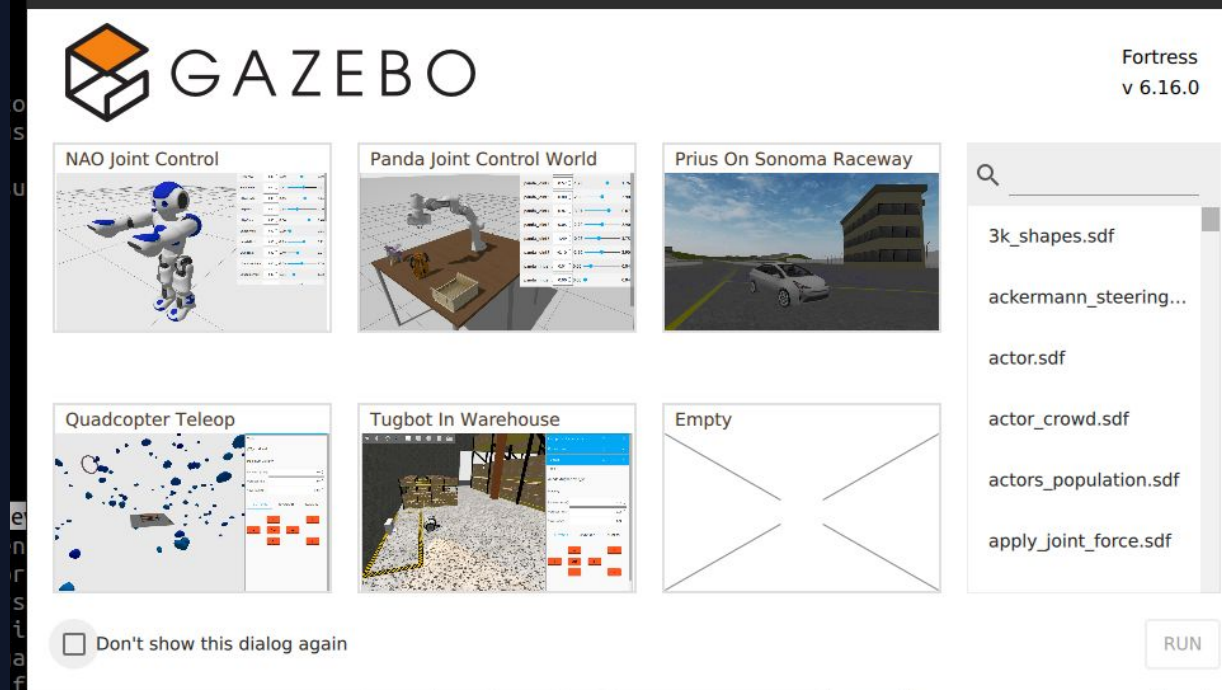
- Fácil de usar y ligero
- Multi-plataforma
- Grandes entornos

ECOSISTEMA DE SIMULACIÓN: GAZEBO

```
ros2 launch ros_gz_sim gz_sim.launch.py
```

```
ros2 launch ros_gz_sim gz_sim.launch.py gz_args:=empty.sdf
```

```
ros2 launch ros_gz_sim gz_server.launch.py world_sdf_file:=empty.sdf
```



HARDWARE Y MICRO-ROS



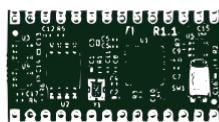
MICRO-ROS

Lleva la potencia de ROS 2 a microcontroladores de bajo consumo.

- > **Micros:** ESP32, Arduino, STM32, Raspberry Pi Pico . . .
- > **Función:** Permite que un sensor de \$5 sea un "Nodo ROS" completo en la red.
- > **Conexión:** WiFi, Serial o Bluetooth al ordenador principal.



PC con ROS2
Micro-ROS Agent



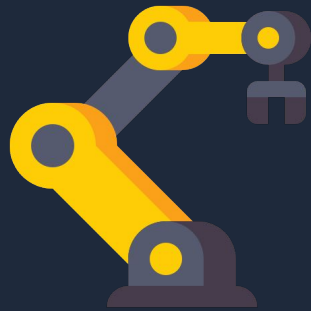
Microcontroller
Micro-ROS Client

TIPOS DE ROBOTS MÁS COMUNES EN ROS



AMR / MÓVILES

Turtlebot, Robots de
almacén, Aspiradoras.
Usan Nav2.



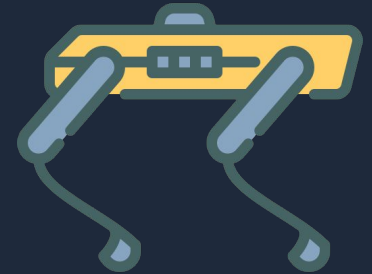
MANIPULADORES

Brazos industriales,
Cobots (UR5, Franka).
Usan MoveIt 2.



AÉREOS / DRONES

Quadcopters, VTOL. Usan
Aerostack2 o PX4
bridge.



CUADRÚPEDOS

Spot, Unitree Go1.
Locomoción compleja y
equilibrio.

INTRODUCCIÓN

CONTROL DE UN PÉNDULO INVERTIDO

Ejemplo del uso ROS2 para una aplicación

- > Estabilizar el péndulo
- > Controlar la posición del carro

```
ros2 launch pendulum_bringup  
pendulum_bringup.launch.py rviz:=True
```

```
ros2 topic pub -1 /teleop  
pendulum2_msgs/msg/PendulumTeleop "cart_position:  
5.0"
```



Robotics Simulation & Control
Dynamics, Algorithms, Real-Time

HERRAMIENTAS

```
ros2 topic list
```

```
ros2 topic echo <topic_name>
```

```
rqt_graph
```

```
rviz2
```

```
ros2 run plotjagger plotjagger
```

```
ros2 bag record -o test --all
```

```
ros2 bag play test
```

INTRODUCCIÓN

TELEOPERACION DEL ROBOT

Control del robot SPOT mediante teleoperación

```
ros2 launch spot_bringup spot.gazebo.launch.py
```

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

```
ros2 launch teleop_twist_joy teleop-launch.py
```

```
ros2 run joy joy_node
```



PARTE 1: ROBÓTICA MÓVIL

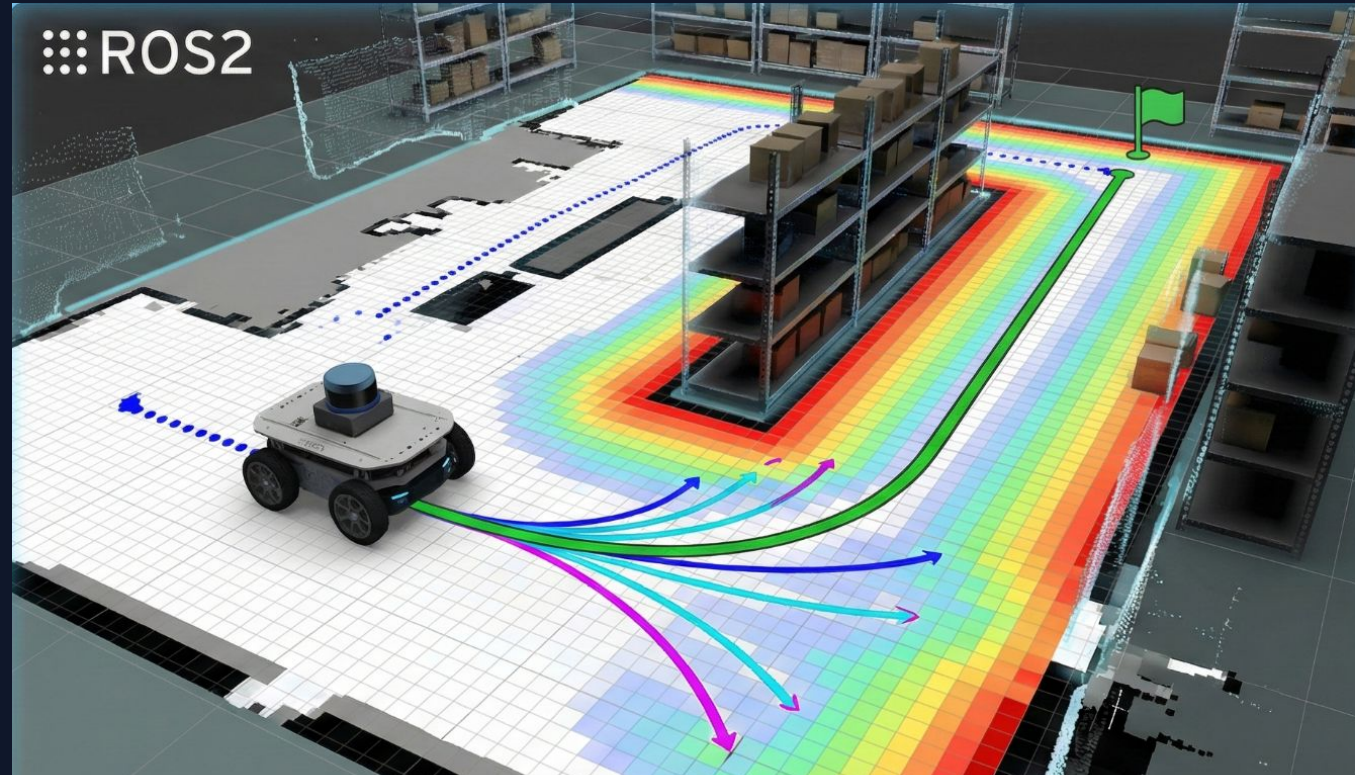
NAVEGACIÓN AUTÓNOMA

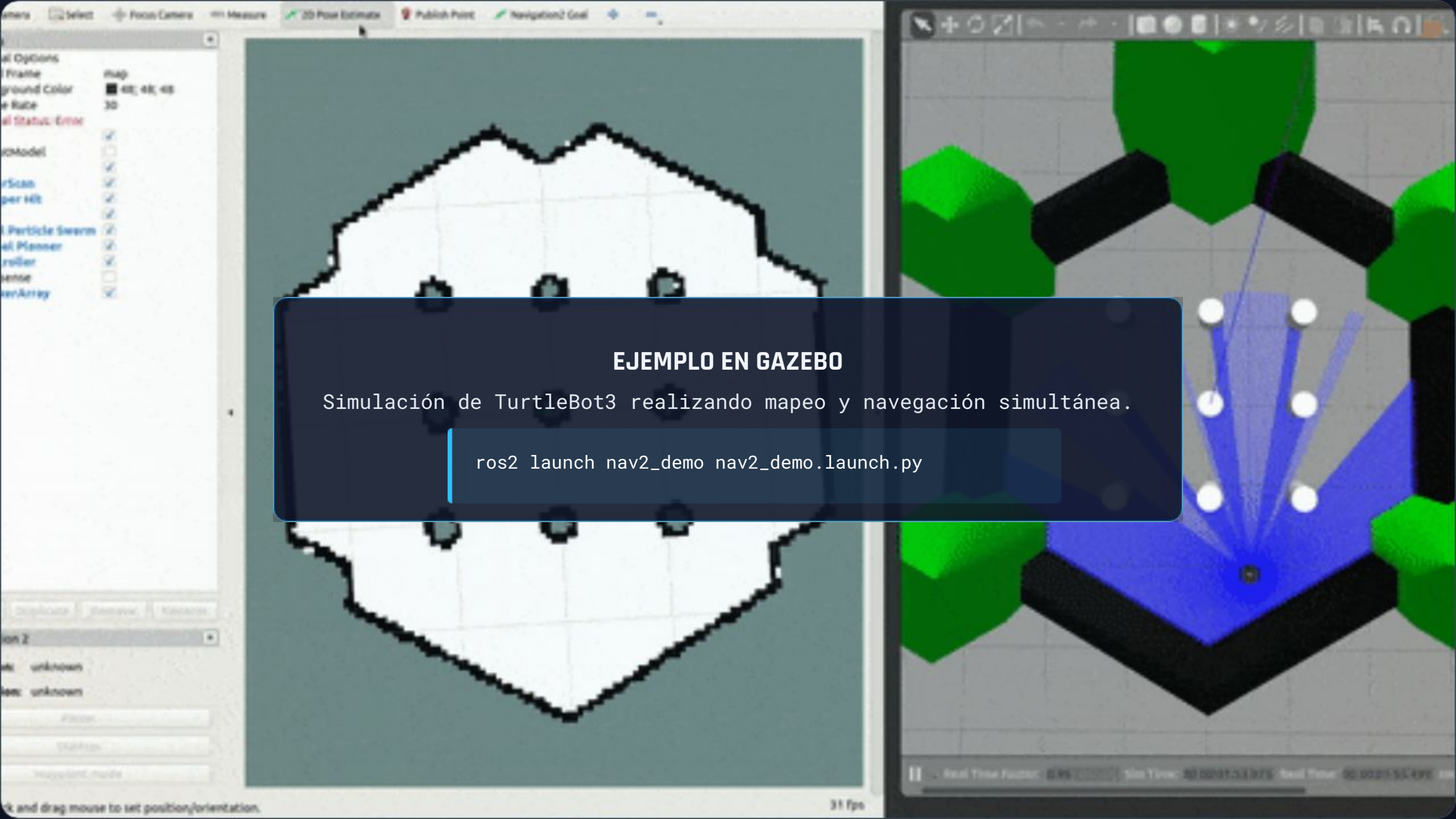
Utilizaremos el stack **Nav2** para mover el robot de forma inteligente.

SLAM

Simultaneous Localization and Mapping.
Construcción de mapa en desconocido.

- > Planificación Global y Local
- > Costmaps (Mapas de costes)





EJEMPLO EN GAZEBO

Simulación de TurtleBot3 realizando mapeo y navegación simultánea.

```
ros2 launch nav2_demo nav2_demo.launch.py
```

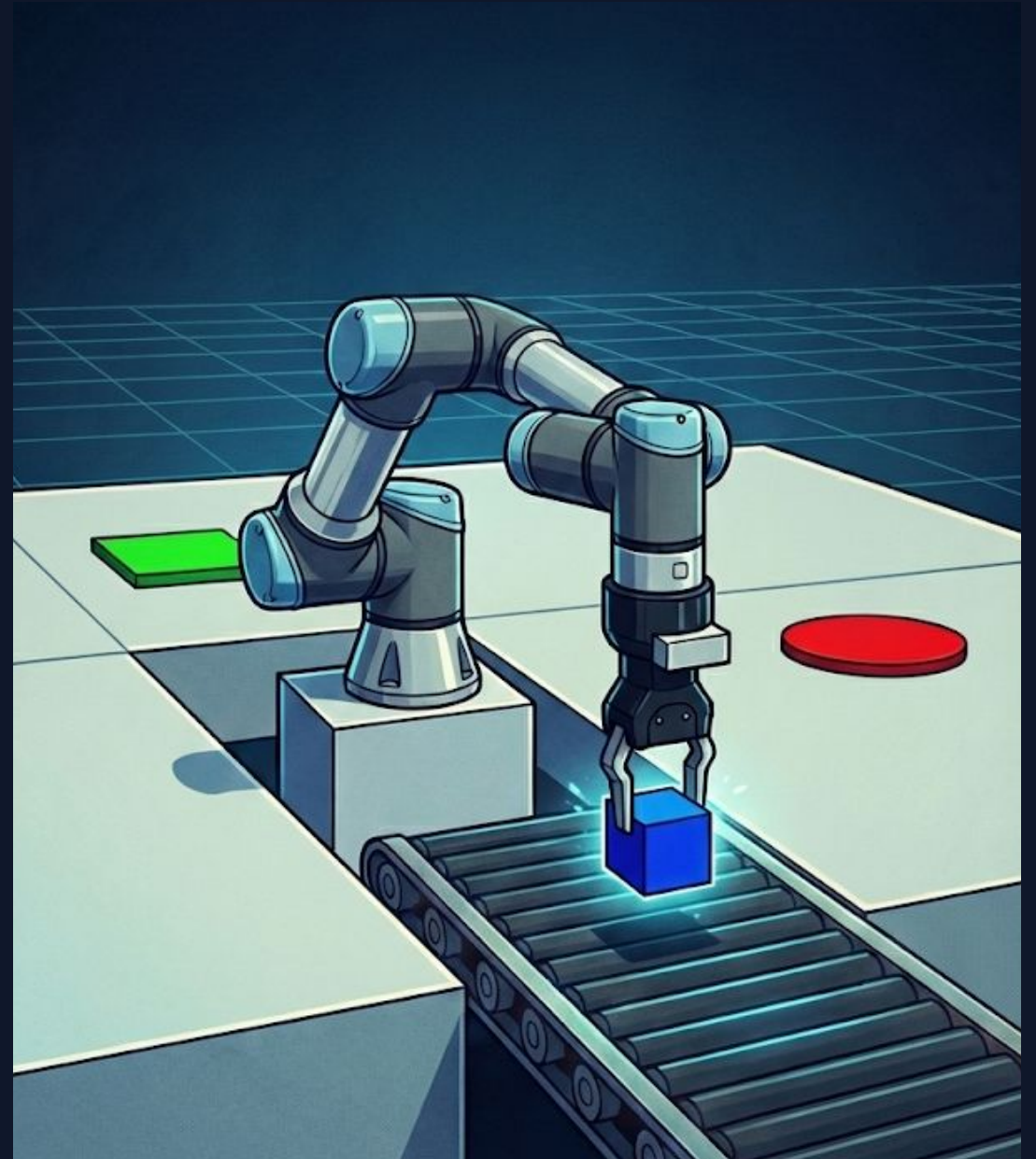
PARTE 2: MANIPULACIÓN

MOVEIT 2

La plataforma estándar para manipulación.

- > Cinemática Inversa y Directa
- > Planificación de Trayectorias (OMPL)
Evasión de colisiones dinámica

```
ros2 launch ros2srcc_launch moveit2.launch.py  
package:=ros2srcc_ur3 config:=ur3_3
```



PARTE 2: MANIPULACIÓN

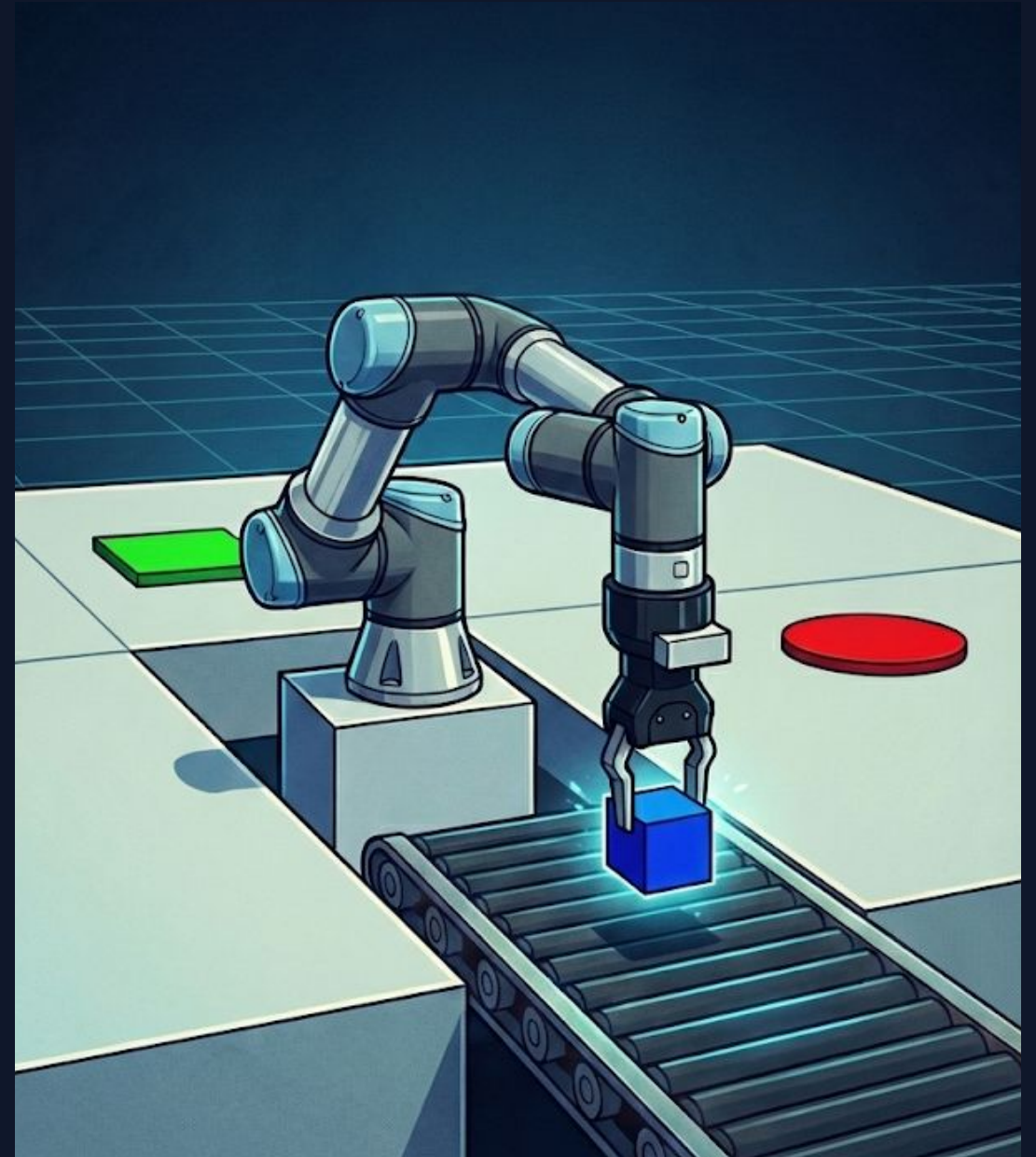
Clasificador de piezas según los vértices

La plataforma estándar para manipulación.

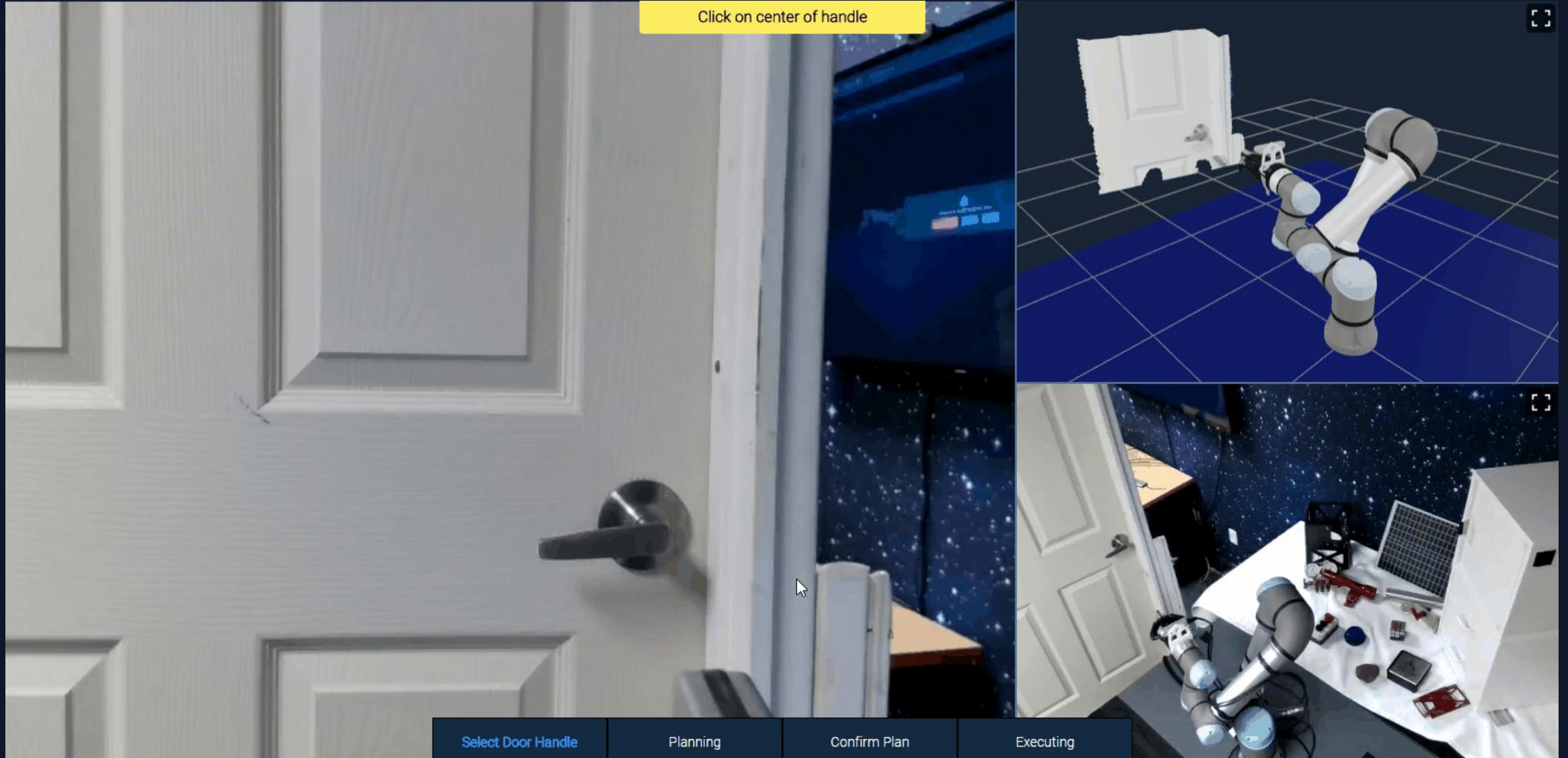
- > Análisis de la pieza usando una cámara
- > Planificación De trayectoria (T0 D0)

```
ros2 launch ros2srrc_launch moveit2.launch.py  
package:=ros2srrc_ur3 config:=ur3_3
```

```
/ros2_ws/src/ros2_SimRealRobotControl/ros2srrc_execution/python$ python3 pick_and_place.py
```



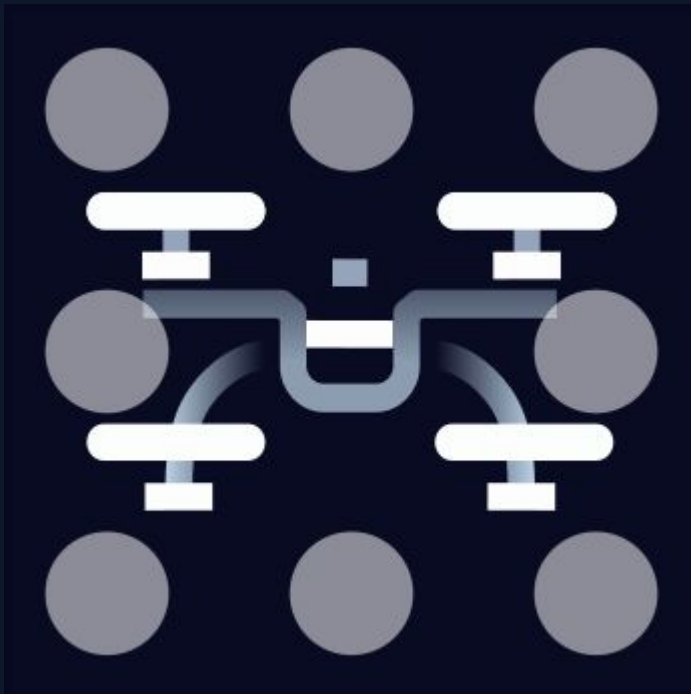
PARTE 2: MANIPULACIÓN



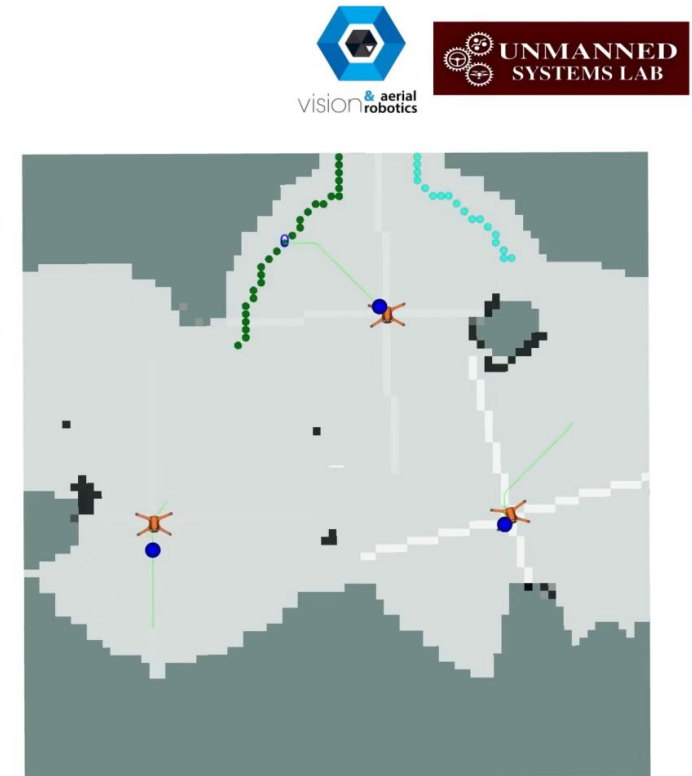
PARTE 3: DRONES Y AEROSTACK2

AEROSTACK2

Framework modular para
sistemas aéreos autónomos.



Experimental results



<https://vimeo.com/893930171?fl=pl&fe=vl>

PARTE 3: DRONES Y AEROSTACK2

```
/ros2_ws/project_gazebo$ ./launch_as2.bash -g -m
```

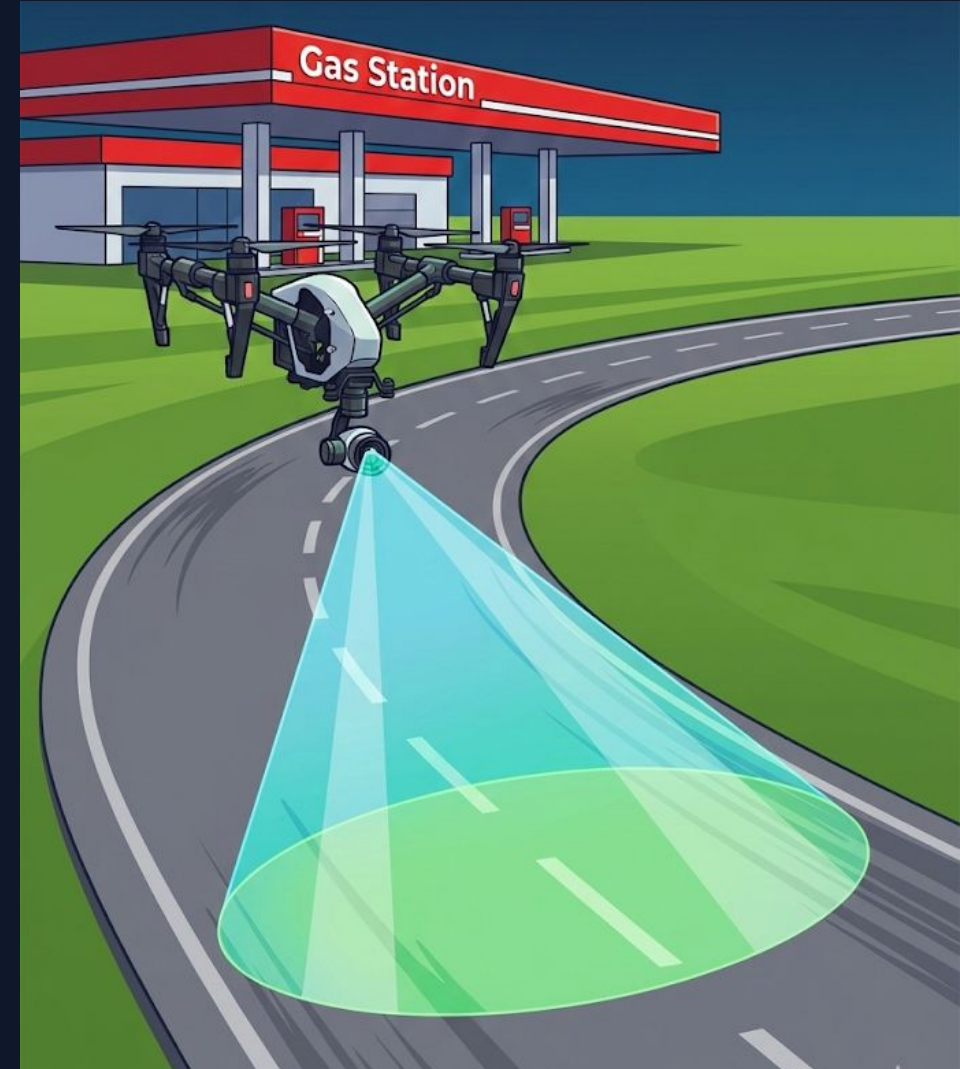
```
/ros2_ws/project_gazebo$ python3 mission_swarm.py
```



PARTE 3: DRONES Y AEROSTACK2

MISIÓN: SEGUIMIENTO DE CARRETERAS

- > Vision por Computador: Deteccion Carretera
- > Control PID: Control sobre la velocidad en yaw
- > Control Fuzzy
- > Control Fuzzy-PID



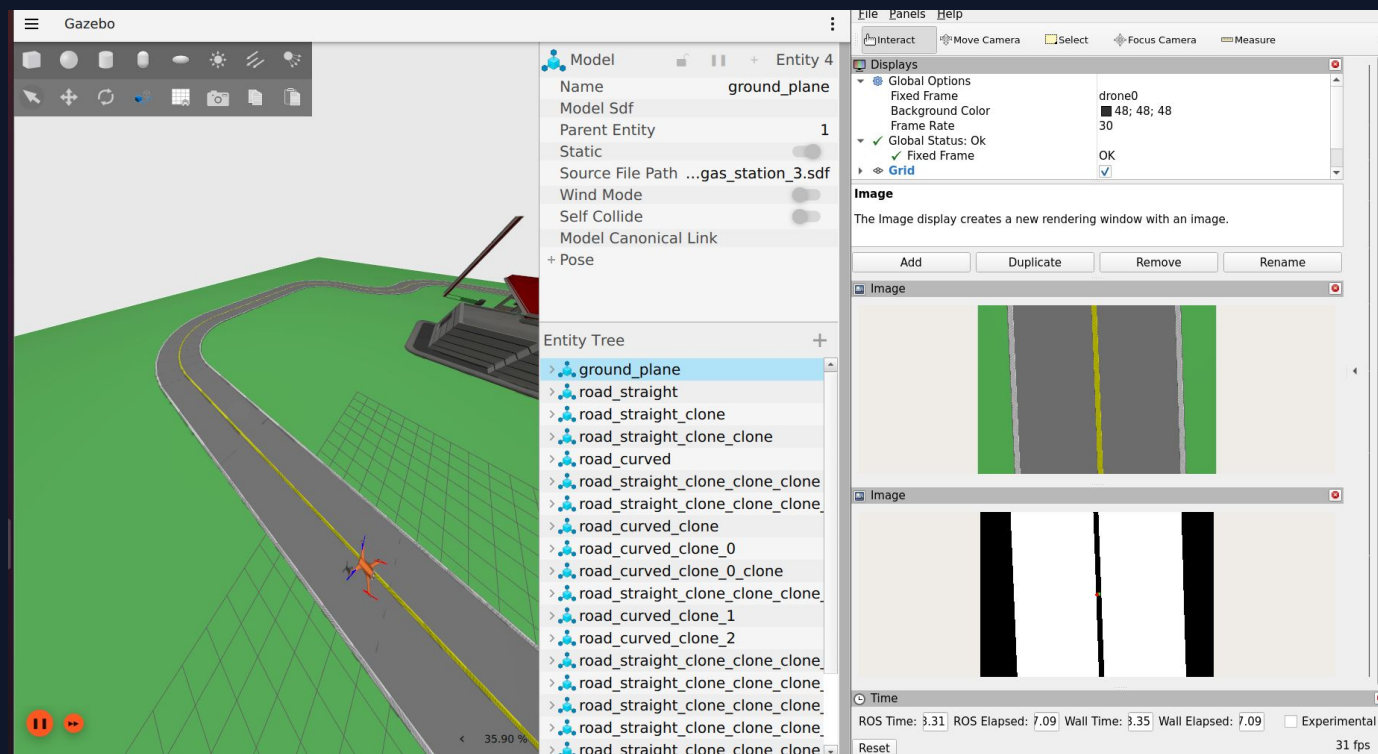
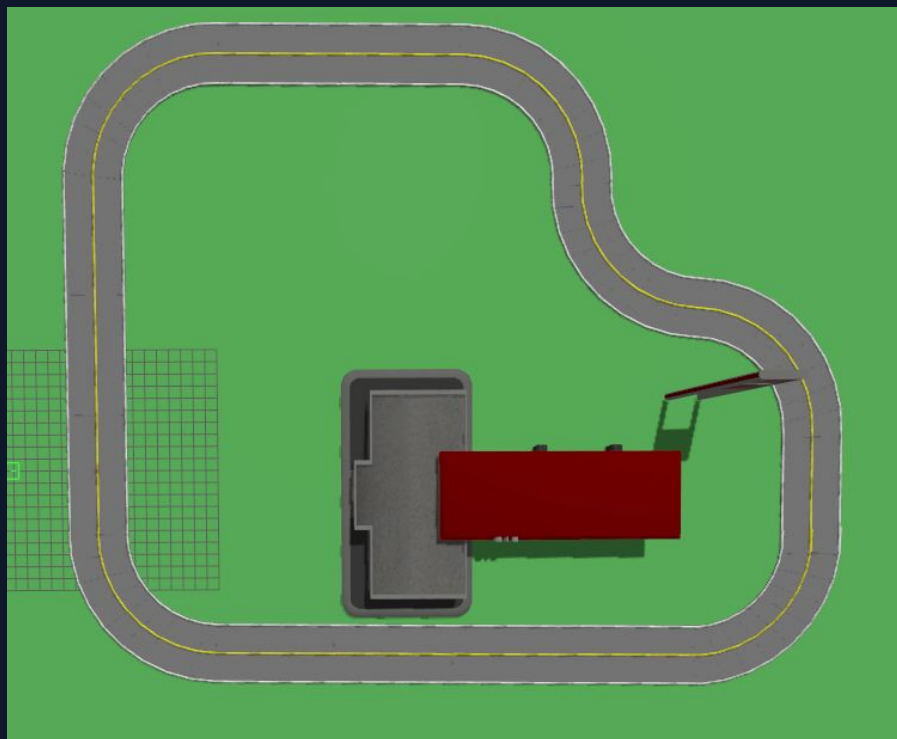
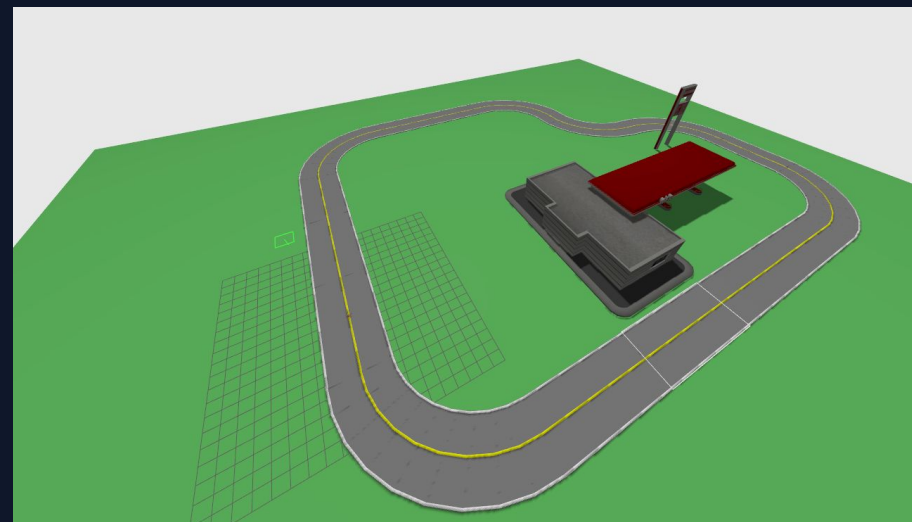
PARTE 3: DRONES Y AEROSTACK2

```
/ros2_ws/demo_road$ ./launch_as2.bash -g
```

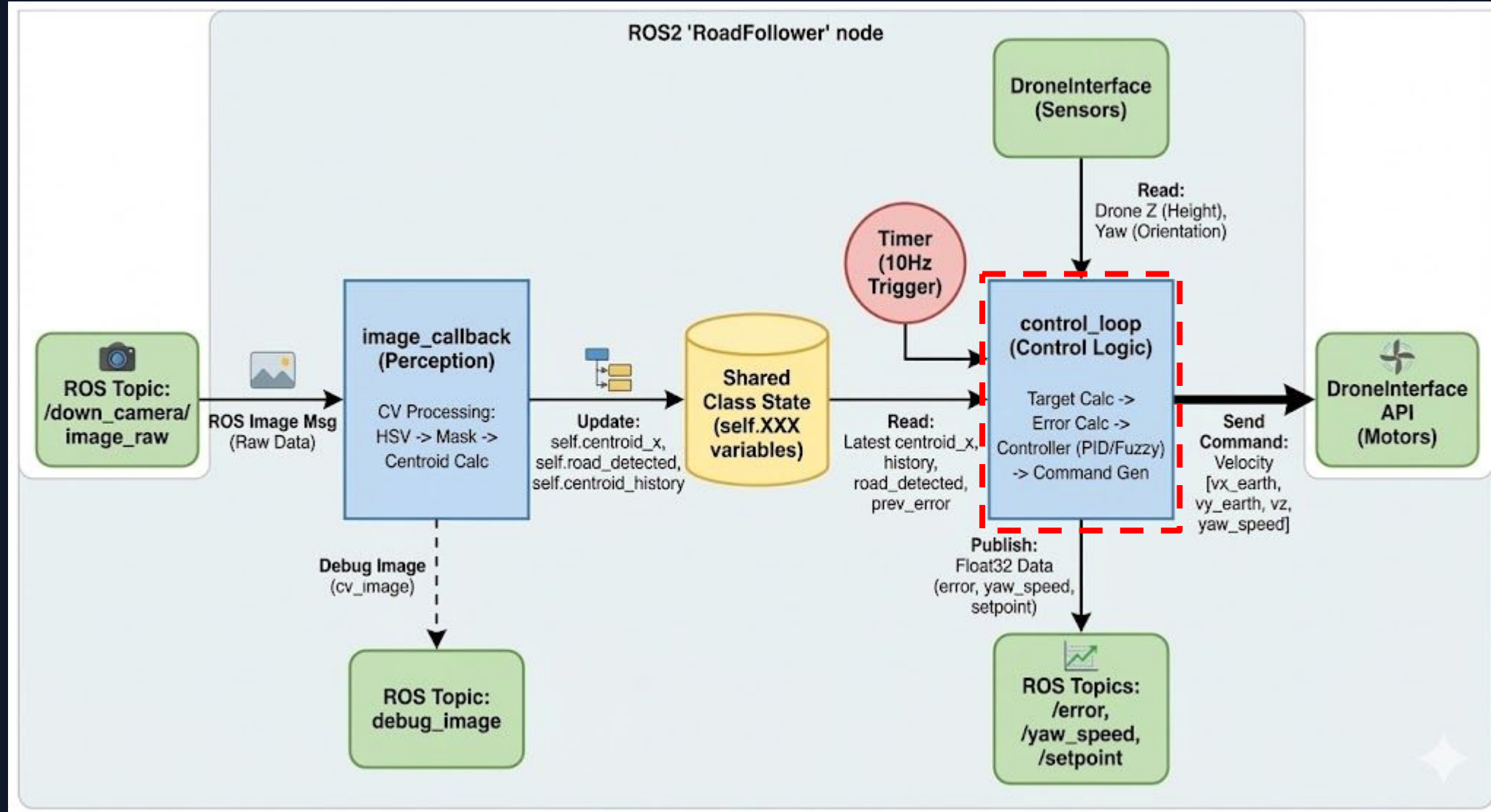
```
/ros2_ws/demo_road$ python3 follow_road.py pid
```

```
/ros2_ws/demo_road$ python3 follow_road.py fuzzy
```

```
/ros2_ws/demo_road$ python3 follow_road.py pid-fuzzy
```



PARTE 3: DRONES Y AEROSTACK2



¿PREGUNTAS?



<https://github.com/davidredondo0/ROS2Curso2>